Water Area

AlgoExpert · Hard · Dynamic Programming

The problem

- Pillars of given heights; how much rain water is trapped?
- Two pointers.

Intuition

- Water above a pillar is limited by the smaller of the tallest walls on its left and right.

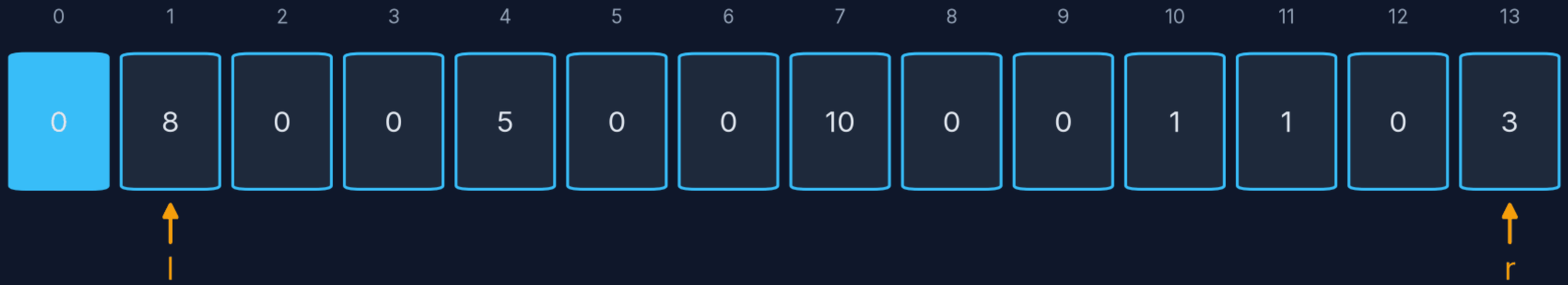
Brute force vs optimal

- Per index: scan left and right for maxes → $O(n^2)$
- Prefix max arrays → O(n) time, O(n) space
- Two pointers → O(n), O(1)

Approach

- 1. Pointers at both ends with running maxes
- 2. Move the side with the smaller max
- 3. Add max minus height
- 4. Continue until they meet

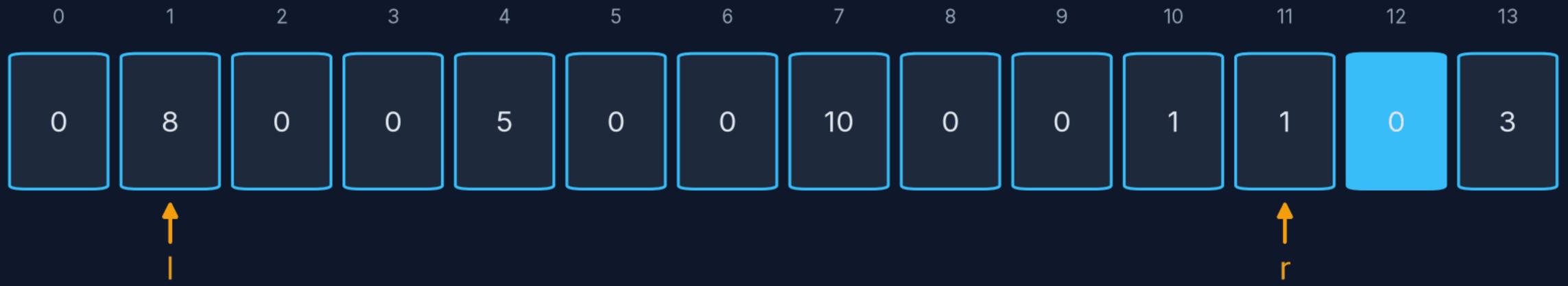
Worked example




```
index 0 holds 0
total = 0
```

Index 0: +0 → 0

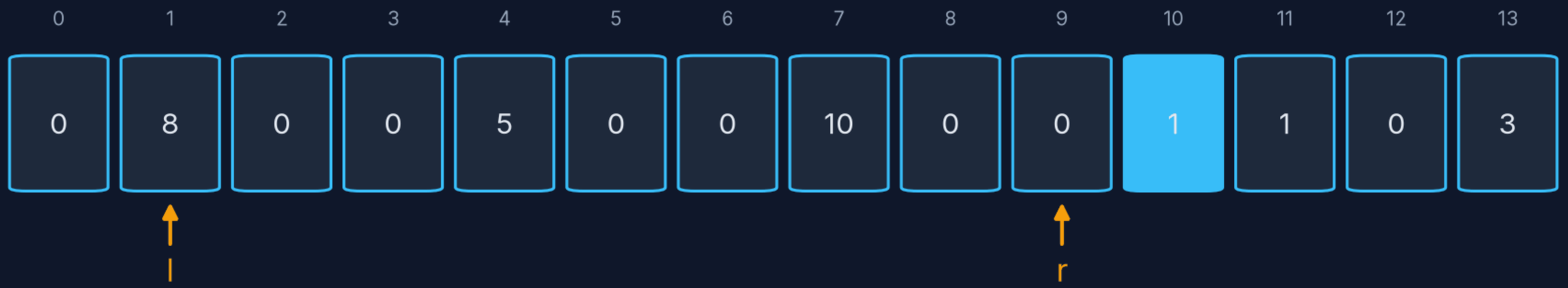
Worked example




```
index 12 holds 3
total = 3
```

Index 12: +3 → 3

Worked example




```
index 10 holds 2
total = 7
```

Index 10: +2 → 7

Worked example




```
index 1 holds 0
total = 13
```

Index 1: +0 → 13

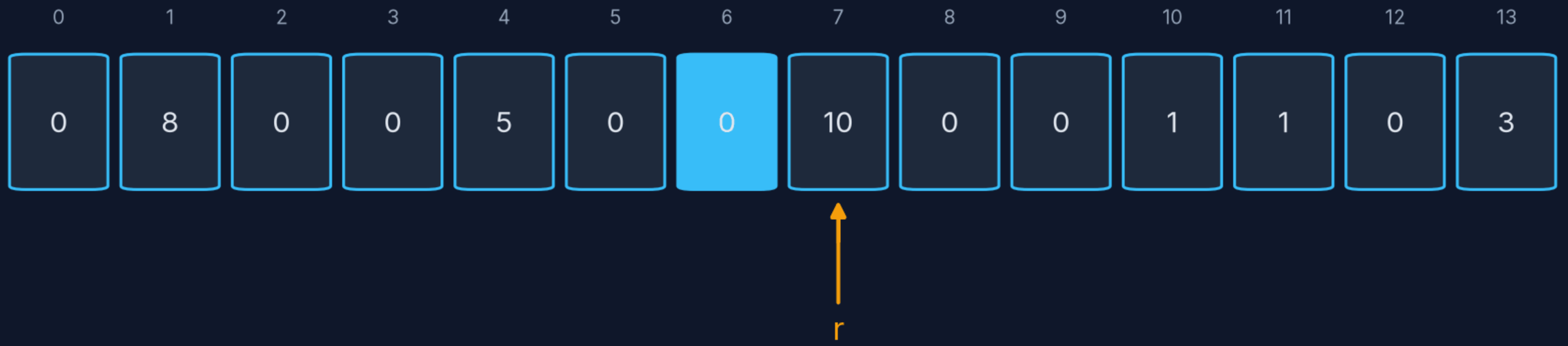
Worked example




```
index 4 holds 3
total = 32
```

Index 4: +3 → 32

Worked example




```
index 6 holds 8
total = 48
```

Index 6: +8 → 48

Java solution

```java
public class WaterArea {
    public static int waterArea(int[] h) {
        int l = 0, r = h.length - 1, lMax = 0, rMax = 0, water = 0;
        while (l < r) {
            if (h[l] < h[r]) { lMax = Math.max(lMax, h[l]); water += lMax - h[l]; l++; }
            else { rMax = Math.max(rMax, h[r]); water += rMax - h[r]; r--; }
        }
        return water;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Empty / short arrays → 0
- Move the lower side
- Water = max − height (never negative)

Interview tips & follow-ups

- Same as Trapping Rain Water

Recap

- Water above a pillar is limited by the smaller of the tallest walls on its left and right.
- Complexity: O(n)